Конспект: Python изнутри (лекции 2–7)

Oct 25, 2026 · @Sokrat

Курс ФТИИ ИТМО, сезон 1, серии 2–7. Сквозная идея: имя — это бирка на объекте, функция — тоже объект, а многое решается ещё до запуска кода.

Карта курса

Лекция	Тема	Главная мысль
S01E02 «Python that I used to know»	Имена и объекты	Переменная — бирка на объекте, а не коробка
S01E03 «Circle of Life»	Копирование и сборка мусора	Объекты образуют граф; копия — это обход графа
S01E04 «Trust Issues»	AST, LEGB, MRO, pytest	Имена раскладывает компилятор до запуска, атрибуты ищутся при каждом обращении
S01E05 «Gotta Catch 'Em All»	Функции изнутри	Функция — объект с «рюкзаком», который собирается на строке def
S01E06 «Корпорация монстров»	Значения по умолчанию и сигнатуры (практикум)	Список появляется там, где выполняется []
S01E07 «Люди в чёрном»	Декораторы	@deco — это просто fn = deco(fn)

Лекция 2. Имена и объекты

Имя — бирка, привязанная к объекту. Тип принадлежит объекту, а имя может указывать на что угодно.

and / or возвращают операнд

```
>>> "ada" and "grace"   # оба истинны → последний
'grace'
>>> "ada" or "grace"    # первый истинный
'ada'
>>> "" or "новичок"
'новичок'
```

- `and` возвращает первый ложный операнд или последний; `or` — первый истинный или последний. Не `True` / `False`.
- `or` не отличает «нет значения» от «значение ложно»: `rating or 1800` заменит и `None`, и `0`. Нужен дефолт только для `None` — пишите `if rating is None`.

Истинность спрашивают у объекта

- Ложны: `0`, `""`, `[]`, `{}`, `None`, `False`. Всё пустое — ложно.
- Истинны: `"0"`, `"False"`, `[0]`, `2` — важно не что внутри, а есть ли внутри хоть что-то.
- `2 == True` → `False` (сравнили с единицей), но `bool(2)` → `True`. Поэтому пишем `if flag:`, а не `if flag == True:`.
- `all([])` → `True`: «на турнир допущена гильдия из нуля человек».

`=` перевешивает бирку, `is` сравнивает объекты

```
>>> guild = ["ada", "grace"]
>>> backup = guild          # вторая бирка на тот же список
>>> guild.append("guido")
>>> backup
['ada', 'grace', 'guido']
```

- `==` — «равны ли значения», `is` — «один ли это объект» (сравнивает `id`).
- `int("256") is int("256")` → `True`, а для `257` → `False`: CPython кэширует маленькие целые (от −5 до 256). Это деталь реализации.
- Правило: `is` — только с `None`, для всего остального `==`.

Аргумент функции — тоже бирка

```
def add_player(roster):
    roster.append("turing")   # меняет объект → видно снаружи

def reset(roster):
    roster = []               # перевешивает свою бирку → снаружи ничего
```

Функция может изменить объект, но не может перевесить чужую бирку.

Изменяемые и неизменяемые

Неизменяемые	Изменяемые
int, float, bool, str, tuple, frozenset, None	list, dict, set

- `ratings["ada"] = 1500` меняет сам словарь — все бирки видят изменение. `rating = 1500` перевешивает бирку — `snapshot` остаётся старым.
- Тест через `id`: `id` изменился — это другой объект (`s += "_Angel"` для строки); `id` тот же — изменили сам объект (`lst.append`).
- Кортеж неизменяем, но список внутри — изменяем: `t = ([1, 2], 3); t[0].append(4)` → `([1, 2, 4], 3)`.
- `range` — объект, который притворяется списком: `sys.getsizeof(range(1_000_000))` = 48 байт, а `list(range(...))` ≈ 8 МБ.

Лекция 3. Копирование и сборка мусора

Объекты ссылаются друг на друга и образуют граф. Копирование — это обход части графа, а сборка мусора — поиск узлов, до которых нельзя добраться.

```
a = []
a.append(a)
print(a)   # [[...]] — интерпретатор знает, что уже был здесь
```

Поверхностная и глубокая копия

	`copy.copy` (поверхностная)	`copy.deepcopy` (глубокая)
Что копируется	Один узел; ссылки ведут в старые объекты	Весь достижимый граф

	`copy.copy` (поверхностная)	`copy.deepcopy` (глубокая)
Риск	Изменение вложенного объекта видно в оригинале	Скопировали игрока — а заодно всю гильдию; дорого

- «Правильного» способа нет — зависит от задачи. Пример: характеристики и вещи игрока копируем, гильдию оставляем общей.
- Мутабельность — свойство узла, а не поддерева: неизменяемый кортеж может держать изменяемый список.
- В графе бывают петли. `deepcopy` не зависает, потому что запоминает уже скопированное (словарь `memo`): для `a = {}; a["self"] = a` копия даст `c["self"] is c` → `True`.

Кто удаляет объекты

1. **Счётчик ссылок.** Каждый объект считает входящие стрелки. Ноль — объект удаляется немедленно.
2. **Петли.** Счётчик принципиально не умеет освобождать петли: у объектов в петле счётчик не падает до нуля. Их ищет циклический сборщик `gc`.
3. **Алгоритм gc:**
   a. Скопировать счётчики ссылок (`gc_refs`).
   b. Вычесть стрелки, идущие изнутри проверяемой группы. Осталось больше нуля — на объект ссылаются снаружи.
   c. Обойти граф от тех, у кого осталось > 0: всё достижимое живо. `gc_refs == 0` ещё не значит «мусор». Остальное удаляется.
4. **Поколения.** Проверять всё каждый раз дорого: `gc.get_threshold()` → `(700, 10, 10)` — новые объекты проверяются часто, пережившие сборку переезжают в старшие поколения и проверяются реже.

- Петля живёт, пока на неё ссылаются снаружи. Момент смерти объекта в петле вы не контролируете.
- gc следит только за объектами, которые могут замкнуть петлю: `gc.is_tracked(0)`, `("ada")`, `((1, 2))`, `({"a": 1})` → `False`; `gc.is_tracked([])`, `({"a": []})`, `((1, []))` → `True`.

Слабые ссылки

```
import weakref
cache = weakref.ref(g)
cache() is not None   # True
del g
cache()               # None — слабая ссылка не держала объект
```

Три вида связи: **обычная** (держит объект), **петля** (объекты держат друг друга), **слабая** (не держит). Слабые ссылки нужны для кэша, который не держит объекты живыми.

Лекция 4. Как Python читает код

Конвейер: **текст → токены → дерево (AST) → таблица имён → байткод → выполнение**. Всё до байткода происходит до запуска («в лагере»), выполнение — «в бою».

Загадка: UnboundLocalError

```
spell_slots = 3
def cast_fireball():
    print(spell_slots)   # UnboundLocalError!
    spell_slots -= 1
```

Компилятор увидел в функции присваивание (`Store`) и решил, что `spell_slots` — локальная для всей функции. Решение принято до первого вызова.
Присваивание в любом месте функции делает имя локальным для всей функции.

Акт I. Код — это дерево

```
import ast
tree = ast.parse("damage = base * 2 + bonus")
print(ast.dump(tree.body[0], indent=2))
# Assign(targets=[Name('damage', Store())],
#        value=BinOp(BinOp(Name('base', Load()), Mult(), Constant(2)), Add(),
Name('bonus', Load())))
```

- Узлы: `Assign` (присваивание), `Name` (имя), `BinOp` (бинарная операция), `Constant` (литерал).
- `ctx=Store()` — в имя пишут, `ctx=Load()` — из имени читают. По `Store` компилятор узнаёт локальные имена.

- Форма дерева = порядок вычисления (глубже = раньше). Приоритет операций — это форма дерева; лишние скобки и комментарии в дереве не хранятся.
- `ast.unparse(tree)` — обратно в текст; `ast.walk(tree)` — обход всех узлов; `compile(tree, "<name>", "exec")` + `exec(code, ns)` — исполнить изменённое дерево.
- Пример «нерфа»: в дереве заменили `Mult` на `FloorDiv` → `damage = base // 2 + bonus`, исходники не тронуты. **Код — это данные.**

Акт II. Компилятор решает, где искать имя

- `fn.__code__.co_varnames` — локальные (у каждой свой номер ячейки); `co_names` — имена, которые ищут в словарях (глобальные, встроенные, атрибуты); `co_freevars` — из объемлющей функции.
- В байткоде (`dis`): `LOAD_GLOBAL` — искать в словаре модуля; `LOAD_FAST_CHECK` — взять локальную по номеру и проверить, что в ней что-то есть.
- Модуль `symtable` показывает решение компилятора: `.is_global()`, `.is_local()`.

Порядок поиска имени LEGB:

Буква	Где	Пример
L — local	Ячейки функции, номера выбраны до запуска	`roll` внутри `cast(roll)`
E — enclosing	Общая ячейка (cell) внешней функции	`base` из `make_spell(base)`
G — global	Словарь имён модуля	`spell_mod = 3`
B — builtins	Встроенные имена	`len`, `sum`, `print`

- Глобальное имя закрывает встроенное: `sum = 20; sum(rolls)` → `TypeError: 'int' object is not callable`; после `del sum` снова работает.
- `global x` / `nonlocal x` — подсказка компилятору: присваивание пишет в глобальную / в ячейку объемлющей функции.
- **Замыкание держит ячейку, а не значение:** `[lambda: i for i in range(3)]` даст `[2, 2, 2]`; исправление — `lambda i=i: i`.

Акт III. Всё есть объект, атрибуты ищутся по MRO

- У каждого значения есть тип, даже у `None` и функции. PyObject в обычной сборке хранит только счётчик ссылок и указатель на тип (`sys.getrefcount`).
- Поиск атрибута `obj.x`:
  1. `obj.__dict__` — словарь самого объекта;
  2. `type(obj)`;
  3. дальше по `type(obj).__mro__`;
  4. `object` — последним; не нашли — `AttributeError`.
- `bool.__mro__` = `(bool, int, object)` — `bool` частный случай `int`, поэтому `True + True == 2` и `sum([True, False, True]) == 2`.
- **Переменные раскладывает компилятор до запуска. Атрибуты ищутся при каждом обращении.**

Эпилог. assert, pytest, mock.patch

- `assert` выключается флагом `python -O`: проверок в байткоде нет. Не проверяйте им входные данные.
- pytest переписывает AST тестов при импорте (хук `AssertionRewritingHook` в `sys.meta_path`): каждая часть выражения кладётся во временную переменную `@py_assert...`. Символ `@` недопустим в идентификаторах — с вашими именами не столкнётся.
- Поэтому pytest показывает `assert 13 == 12` + `where 13 = damage([6, 4], 3)`: для локальных имён — значение, для глобальных функций — имя. В переписанных `.pyc` есть тег `-pytest-9.1.1`. Для вспомогательных модулей — `pytest.register_assert_rewrite("helpers")` в `conftest.py`.
- **mock.patch: патчите там, где имя ищут, а не там, где определили.**

```
# attack_a.py:  import random;  random.randint(1, 20)   → атрибут ищется при
каждом вызове
# attack_b.py:  from random import randint            → своё имя в глобальных
attack_b
with patch("random.randint", return_value=20):
    attack_a.attack(), attack_b.attack()   # крит!, мимо
with patch("attack_b.randint", return_value=20):
    attack_b.attack()                      # крит!
```

- Мутационное тестирование: заменили в AST `>=` на `>` — тесты зелёные, мутант выжил. Значит, тесты не проверяют границу (`hits(12, 12)`).

Лекция 5. Функции изнутри

Функция — обычный объект («карточка») с «рюкзаком», который собирается один раз на строке `def`. Важно различать три момента: **до запуска**, **в момент def**, **во время вызова**.

Акт I. Карточка и рюкзак

- У функции есть `__name__`, `__doc__` (docstring), `__module__`, `__code__.co_varnames`.
- Нет `return` — возвращается `None`. `return` сразу завершает функцию.
- `return hp, hp > 0` возвращает **один кортеж**, его распаковывают: `hp, alive = fight(35, 10)`.
- Аргументы: сначала по порядку, потом по имени. `attack(pokemon="X", "Ember")` → `SyntaxError: positional argument follows keyword argument`.
- Функция — значение: можно положить в словарь и вызвать по ключу: `moves["ember"]("Pidgey")`.
- **Frame** (кадр вызова) живёт один вызов: `battle()` с `hp = 100; hp -= 30` всегда даёт 70. Локальные не становятся атрибутами функции (`hasattr(battle, "hp")` → `False`). Посмотреть: `inspect.currentframe().f_locals`.

Четыре кармана рюкзака = четыре буквы LEGB:

Буква	Где лежит	Когда появляется
L	Кадр вызова, `f_locals`	Во время вызова
E	Ячейки-«банки», `__closure__` (кортеж cell)	В момент def
G	Словарь родного модуля, `__globals__`	В момент def
B	Словарь встроенных, `__builtins__`	В момент def

- Где искать каждое имя, решено до запуска. Значение присваивается во время выполнения.
- **Глобальные имена функция ищет в своём модуле.** `from pallet_town import greet` + `REGION = "Johto"` в main.py → `greet()` всё равно вернёт Kanto: `greet.__globals__ is pallet_town.__dict__`.
- Вызов функции — это присваивание аргументов параметрам.

Акт II. Лямбды и замыкания

- `lambda p: p["level"]` — та же функция, что и `def`: тот же тип и байткод, только имя `<lambda>`.
- Хороша как одноразовый `key=`: `sorted(team, key=lambda p: p["level"])`. В трассировке ошибки видно лишь `in <lambda>`.
- Нужно имя — пишите `def`, а не `by_level = lambda ...` (PEP 8, ruff E731).
- Лямбды в цикле держат одну ячейку: `[lambda: s for s in names]` → три раза последнее имя. `stages[0].__closure__[0] is stages[1].__closure__[0]` → `True`.
- `nonlocal` пишет в ячейку — так делают счётчик с памятью:

```
def pokedex():
    caught = 0
    def catch(name):
        nonlocal caught
        caught += 1
        return f"{name}: поймано {caught}"
    return catch

catch = pokedex()
catch("Pidgey")    # поймано 1
catch("Rattata")   # поймано 2
```

Акт III. Значения по умолчанию вычисляются один раз

- Вычисляются, когда выполняется `def`, и хранятся в `fn.__defaults__`.
- `def wild(level=random.randint(1, 100))` — уровень выпал один раз и одинаков при каждом вызове.
- Починка лямбд в цикле: `lambda s=s: s` — у каждой свой `__defaults__`, `__closure__` = `None`.
- Классическая ловушка `def add_to_party(pokemon, party=[])`: список один на все вызовы, `ash is misty` → `True`. Правильно:

```
def add_to_party(pokemon, party=None):
    if party is None:
        party = []          # новый список при каждом вызове
    party.append(pokemon)
    return party
```

Акт IV. Сигнатуры: пять зон параметров

```
def attack(pokemon, /, move, *targets, crit=False, **effects): ...
```

Параметр	Вид (`inspect.Parameter.kind`)	Что принимает
`pokemon` (до `/`)	POSITIONAL_ONLY	Только по позиции (PEP 570)
`move`	POSITIONAL_OR_KEYWORD	По позиции или по имени
`*targets`	VAR_POSITIONAL	Все лишние позиционные → tuple
`crit` (после `*`)	KEYWORD_ONLY	Только по имени (PEP 3102)
`**effects`	VAR_KEYWORD	Все лишние именованные → dict

- `*targets` ловит всё: `attack("Pikachu", "Thunderbolt", "Onix", True)` → `targets = ('Onix', True)`, `crit = False`. Флаг нужно передавать по имени.
- Голая `*` — охранник: `def attack(p, m, *, crit=False)`; `attack(p, m, True)` → `TypeError`.
- До `/` имена не принимаются, поэтому `attack("Pikachu", pokemon="Meowth")` при `**effects` даст `('Pikachu', {'pokemon': 'Meowth'})`.
- **В def собирают, при вызове раскладывают:** `attack("Charmander", "Ember", *team, **status)`.
- `inspect.signature(fn).bind(...)` показывает, куда разложатся аргументы, без вызова функции.

Лекция 6. Смена в Корпорации монстров (практикум)

Лекция закрепляет темы L05 на задачах. Итог смены: **журнал появляется там, где выполняется `[]`; сигнатура определяет, куда попадут аргументы; допустимость значений проверяет код программы.**

Замыкания-счётчики

Каждый вызов `make_counter()` создаёт новую ячейку `count`: `mike()` → 1, 2, а `sulley()` → 1. У каждого счётчика своя ячейка.

Когда создаётся список по умолчанию

- `def enter(monster, log=prepare_log())` печатает `PREPARE` до `READY`: дефолт вычислен на строке `def`, до любого вызова.
- Между вызовами список живёт в `enter.__defaults__[0]`: `one is two is enter.__defaults__[0]` → `True`.
- Если `def` вложен в фабрику (`make_station`), каждый вызов фабрики снова выполняет `def` → у каждой функции свой список. Но внутри одной функции вызовы всё равно накапливаются: `a("Sulley")`, `a("Mike")` → `['Sulley', 'Mike']`, `b("Roz")` → `['Roz']`.
- Дефолт сохраняет ссылку на объект, а не имя: `read = lambda saved=log: saved`, потом `log.append("A7")` и `log = ["B8"]` → `read()` вернёт `['A7']`.

`log or []` — скрытая ошибка

```
def enter(monster, log=None):
    if log is None:       # правильно
        log = []
    # log = log or []     # ошибка: переданный ПУСТОЙ список ложен и будет
подменён новым
    log.append(monster)
    return log

provided = []
enter("Roz", provided) is provided   # True только с is None
```

Диспетчерская: *args, keyword-only, positional-only

- `dispatch(monster, *doors)`: `dispatch("Mike")` → `doors = ()` (пустой кортеж, не ошибка).
- `dispatch(monster, *doors, urgent=False)`: `dispatch("Sulley", "A7", True)` → `(('A7', True), False)` — `True` уехал в двери. Нужно `urgent=True`.
- `dispatch(monster, /, door)`: `dispatch(monster="Mike", door="A7")` → `TypeError`, тело не выполняется.
- `pack(*doors)` создаёт новый кортеж, но элементы — те же объекты: `a is b` → `False`, `a[0] is b[0]` → `True`.

inspect.signature: bind проверяет форму, не значения

```
from inspect import signature
bound = signature(dispatch).bind("Sulley", "A7", True)   # тело НЕ
выполняется
bound.arguments          # {'monster': 'Sulley', 'doors': ('A7', True)}
bound.apply_defaults()   # добавит urgent=False, effects={}
dispatch(*bound.args, **bound.kwargs)   # вызов с теми же аргументами
```

- `signature(fn).parameters` — параметры с `.kind` (пять видов из L05).
- Успешный `bind` значит лишь, что аргументы разложились по параметрам. Что дверь — строка, проверяет тело функции: `if not isinstance(door, str): raise ValueError(...)`.

Домашние вопросы со слайдов

- [ ] Разделить журналы в `make_shift_log(log=[])`: журнал создаётся в вызове фабрики (`log=None` → `log = []`), `record` держит его через `__closure__`; явно переданный пустой журнал сохраняется; написать тест против исходной ошибки.
- [ ] Исправить `add(item, log=[])` так же.
- [ ] `send("A7", True)` при `def send(*doors, urgent=False)` — исправить вызов на `send("A7", urgent=True)`.

Лекция 7. Декораторы («Люди в чёрном»)

Декоратор — функция, которая принимает функцию и возвращает обёртку. Запись `@guard` над `def fire` — ровно `fire = guard(fire)`.

Основа

- Имя — бирка: `backup = fire; fire = None; backup()` всё ещё стреляет. Функцию можно передать и вернуть.
- Проверка должна идти **до** вызова: если сначала `fire(agent)`, а потом `return "denied"`, выстрел уже случился.

```
from functools import wraps

def guard(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        agent = args[0]
        if agent["clearance"] < 2:
            return "denied"           # fn не вызывается
        return fn(*args, **kwargs)    # распаковать и ВЕРНУТЬ результат
    return wrapper

@guard
def fire(agent):
    agent["shots"] += 1
    return "ok"
```

- Обёртка — замыкание: `safe_fire.__code__.co_freevars` → `('fn',)`, `__closure__[0].cell_contents is fire` → `True`.
- Декоратор срабатывает **один раз, при определении** функции; `wrapper` — при каждом вызове. Порядок: `decorate`, `ready`, `enter`, `БАХ!`, `enter`, `БАХ!`.

Типичные ошибки в обёртке

Ошибка	Что происходит	Как правильно
`fn(args, kwargs)`	Функция получает кортеж и словарь как два аргумента	`fn(*args, **kwargs)`
Нет `return` в wrapper	Результат теряется, возвращается `None`	`return fn(*args, **kwargs)`
Нет `@wraps(fn)`	`__name__` = `wrapper`, `__doc__` = `None`, `signature` = `(*args, **kwargs)`	`@wraps(fn)`: копирует имя и docstring, добавляет `__wrapped__`

- Исключение из `fn` проходит сквозь обёртку без изменений (тот же объект).
- `inspect.signature` ходит по `__wrapped__`: с `@wraps` покажет `(parcel, *, urgent=False)`, и `bind()` без аргументов упадёт `TypeError`.
- Аннотации (`value: int`) ничего не проверяют: `clearance("trainee")` спокойно вернёт строку.

Декоратор с параметрами (три уровня)

```
def requires_clearance(min_level):          # 1. настройка
    def decorate(fn):                       # 2. оборачивание
        @wraps(fn)
        def wrapper(*args, **kwargs):       # 3. вызов
            bound = signature(fn).bind(*args, **kwargs)
            agent = bound.arguments["agent"]   # работает и для fire(j), и
для fire(agent=j)
            if agent["clearance"] < min_level:
                return "denied"
            return fn(*args, **kwargs)
        return wrapper
    return decorate

@requires_clearance(2)      # = fire = requires_clearance(2)(fire)
def fire(agent): ...
```

- У каждого настроенного декоратора своя ячейка `min_level`: `requires_clearance(2)` и `(4)` не мешают друг другу.
- Проверка внутри `wrapper` идёт при каждом вызове — повысили `clearance` агенту, и доступ появился.
- Антипример `frozen_guard`: если вычислить `allowed` в момент оборачивания, обёртка будет помнить «вчерашнее разрешение» и пустит агента без допуска.

Несколько декораторов: порядок важен

```
@glasses
@flash
def report(): ...
# = report = glasses(flash(report))
```

- Нижний декоратор применяется первым, но снаружи оказывается верхний. При вызове: enter glasses → enter flash → report → exit flash → exit glasses.
- Пример с аудитом: `audit(requires_clearance(2)(action))` запишет отказ `['denied']`, а `requires_clearance(2)(audit(action))` — ничего: внутренний аудит не был вызван. Аудит должен быть снаружи, чтобы видеть весь вызов.

Декоратор-журнал и исключения

```
def audit(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        log = signature(fn).bind(*args, **kwargs).arguments["log"]   # НЕ "or
[]"
        try:
            result = fn(*args, **kwargs)
        except Exception as error:
            log.append(("error", type(error).__name__))
            raise                 # без аргумента — поднять то же исключение
        log.append(result)
        return result
    return wrapper
```

- `bound.arguments["log"] or []` ломает журнал: переданный пустой `journal` подменяется новым списком, и запись теряется.
- `except TypeError` вокруг `bind` ловит только неверные аргументы; `TypeError` из тела обработчика туда не попадёт.
- Три момента жизни декоратора: **создание декоратора → вызов обёртки → возврат или исключение**.

Шпаргалка: ловушки курса

Ловушка	Что получится	Почему	Как правильно	Лекция
`def f(x, acc=[])`	Вызовы делят один список	Дефолт вычислен один раз, лежит в `__defaults__`	`acc=None` + `if acc is None: acc = []`	5, 6
`x = x or []`	Переданный пустой список подменён	Пустой список ложен	`if x is None`	2, 6, 7
`[lambda: i for i in ...]`	Все возвращают последнее значение	Замыкание держит ячейку, а не значение	`lambda i=i: i`	4, 5, 6
Чтение, потом `x -= 1` в функции	`UnboundLocalError`	Присваивание делает имя локальным для всей функции	`global` / `nonlocal` или вернуть новое значение	4
`x is 257`	Непредсказуемо	`is` сравнивает объекты, малые int кэшируются	`==`; `is` только с `None`	2
`if flag == True:`	`2 == True` → `False`	Сравнение с единицей, а не истинность	`if flag:`	2
`b = a` для списка	Изменение видно через оба имени	`=` вешает вторую бирку	`copy.copy` / `copy.deepcopy` по задаче	2, 3

Ловушка	Что получится	Почему	Как правильно	Лекция
`f(a, b, True)` при `def f(*xs, flag=False)`	`True` уходит в `xs`	`*xs` собирает все позиционные	`f(a, b, flag=True)`	5, 6
`patch("random.randint")` при `from random import randint`	Патч не срабатывает	У модуля своя ссылка на функцию	Патчить там, где ищут: `patch("mod.randint")`	4
`assert` для проверки данных	При `python -O` проверка исчезает	assert вырезается из байткода	`if ...: raise ValueError`	4
Обёртка без `return` / `@wraps`	`None` вместо результата, имя `wrapper`	Результат и метаданные не проброшены	`@wraps(fn)` + `return fn(*args, **kwargs)`	7
Аннотация как проверка	`f("trainee")` при `value: int` работает	Аннотации только описывают	`isinstance` в теле	7

Конспект составлен по файлам L02.pdf, L03.pdf, L04.pdf, L05v2.pdf, lecture06-keynote.pdf, lecture07-mib.pdf. В L02 и L03 часть слайдов — картинки без текста, здесь передан их смысл.